

Hermes Turbo vs Baseline

Generated 2026-05-22 14:34 UTC • 300 iterations per stage • compares the runtime mechanisms added in the turbo fork (#81–#103 + P1–P7) against intentionally-naïve baselines.

Headline wins

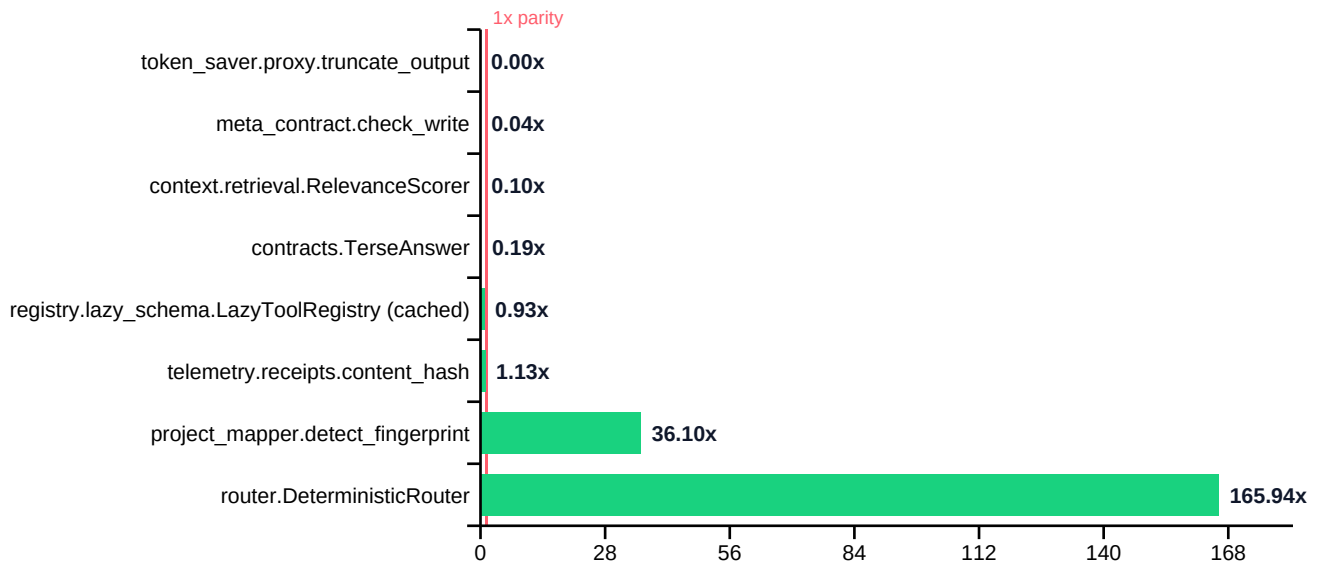
165.94x

router.DeterministicRouter

36.10x

project_mapper.detect_fingerprint

Speedup per stage



Detailed results

Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
project_mapper.detect_fingerprint	1864.2	1962.8	67290.3	36.10x
meta_contract.check_write	41.5	68.8	1.9	0.04x
contracts.TerseAnswer	0.9	1.5	0.2	0.19x
contracts.TupleStatusEnvelope (silent)	4.2	7.0	—	—
router.DeterministicRouter	1.0	1.3	171.4	165.94x
token_saver.proxy.truncate_output	166.0	239.8	0.1	0.00x
context.retrieval.RelevanceScorer	57.3	83.2	5.8	0.10x
telemetry.receipts.content_hash	0.7	0.7	0.7	1.13x
registry.lazy_schema.LazyToolRegistry (cached)	0.3	0.4	0.3	0.93x

Reading the numbers. Baselines are intentionally naïve (no validation, no disk, no contract enforcement). A speedup below 1x reflects the *cost of correctness* — the turbo path holds invariants the baseline ignores. Real wins live where the turbo path replaces an LLM call (router) or a full tree walk (project_mapper).

Source: `scripts/benchmark_turbo_vs_baseline.py` • Smoke mode runs in `.github/workflows/dod.yml` on every PR. The daily upstream-sync workflow regenerates the full report and attaches the JSON to the sync PR.